Death is not Game Over

The Curiquingue Tamagotchi Circuit — Assembly Manual

Introduction to Curiquingue Umbral

Ever wondered how a small screen and a handful of lights can tell a whole story? This project introduces you to Curiquingue Umbral — a hand-built Arduino arcade where a pixel-art Andean raptor descends from the mountains, faces a polluted river, and passes through a threshold instead of simply "losing." In this build, we're using an Arduino Uno, a small OLED display, an analog joystick, and a strip of addressable NeoPixel LEDs — all parts that are affordable and beginner-friendly, even if the story they tell is not simple at all.

Key Components

1. Arduino Uno — controls everything: reads the joystick, draws the screen, and drives the lights.
2. Analog Joystick — lets the player move the curiquingue and jump.
3. SSD1306 OLED Display (128×64) — shows the game itself.
4. 22-LED NeoPixel Strip — lights that change color depending on what is happening in the game.
5. Jumper wires — for making all the connections.
6. Breadboard — to hold everything together without soldering, while prototyping.

Operation

In this circuit:

- The joystick works as a movement controller, the same way a potentiometer does:
  - Moving it left/right changes a voltage on the X axis (0–5V)
  - Moving it up/down changes a voltage on the Y axis (0–5V)
  - Arduino reads both as values from 0–1023
- The OLED shows whichever scene the game is currently in — the descent, the maze, or one of the threshold moments.
- The NeoPixel strip is updated on every loop, and always reflects the current game state — it is never just decorative lighting.
- Arduino redraws the whole screen every loop — there are no saved frames, everything is calculated live.

Required Materials

1. Arduino Uno
   - USB cable / USB-C adapter, if needed
2. Analog Joystick module
3. SSD1306 OLED Display, 128×64, I2C
4. NeoPixel Strip, 22 LEDs (WS2812B)
5. Jumper wires
6. Breadboard

Components

Electronic Component	What it does
Arduino Uno	The board that runs the whole game — reads the joystick, draws the screen, drives the lights.
Analog Joystick module	Two potentiometers (X/Y) plus a click button. Moves the curiquingue and triggers jumps.
SSD1306 OLED display (128×64, I2C)	The screen. Shows the descent, the maze, and every threshold scene.
22-LED NeoPixel strip (WS2812B)	Addressable RGB LEDs. Never just decoration — always tied to the current game phase.

Tools Needed

- Wire strippers
- Computer with Arduino IDE installed

Step-by-Step Assembly Guide

Safety First

- Always disconnect power before making connections
- Avoid touching electronic components directly
- Work on a clean, non-metallic surface

Assembly Steps

Part 1: Initial Setup

1. Place your Arduino on the left side of the breadboard
2. Make sure your workspace is clear and well-lit

Part 2: Joystick Connection

1. Identify the joystick pins: GND, +5V, VRX, VRY, SW
2. Connect the joystick wires:

Connect GND from the joystick to Arduino's GND pin
Connect +5V from the joystick to Arduino's 5V pin
Connect VRX from the joystick to Arduino's A0 pin
Connect VRY from the joystick to Arduino's A1 pin

Note: the click button (SW) is wired but not used for jumping — jumping is read from the Y axis instead, moving the stick up. This was a deliberate change after testing the SW pin's behavior on this board.

Part 3: OLED Display Connection

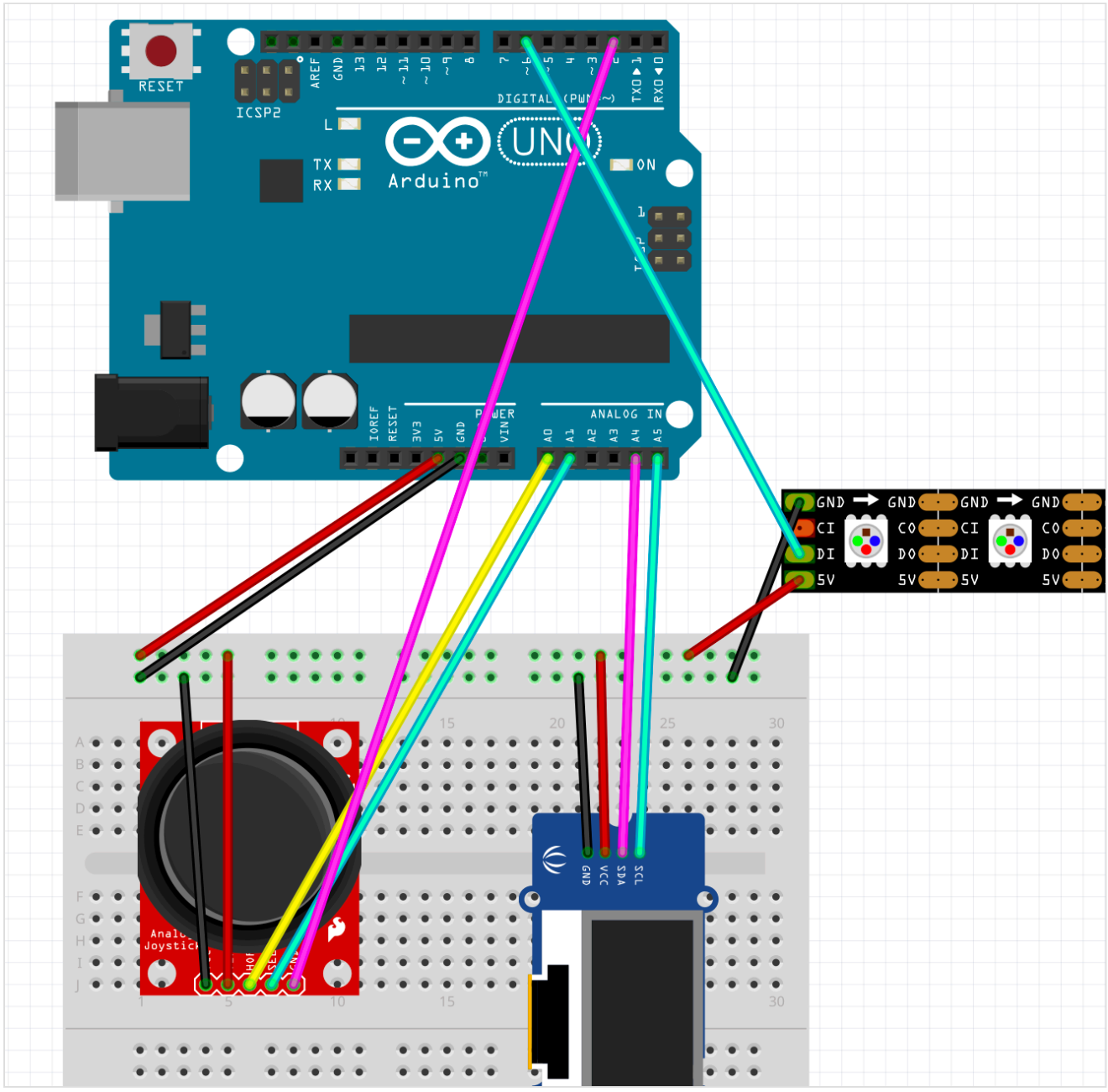
1. Identify the OLED pins: VCC, GND, SCL, SDA
2. Connect the OLED wires:

Connect VCC from the OLED to Arduino's 5V pin
Connect GND from the OLED to Arduino's GND pin
Connect SCL from the OLED to Arduino's A5 pin
Connect SDA from the OLED to Arduino's A4 pin

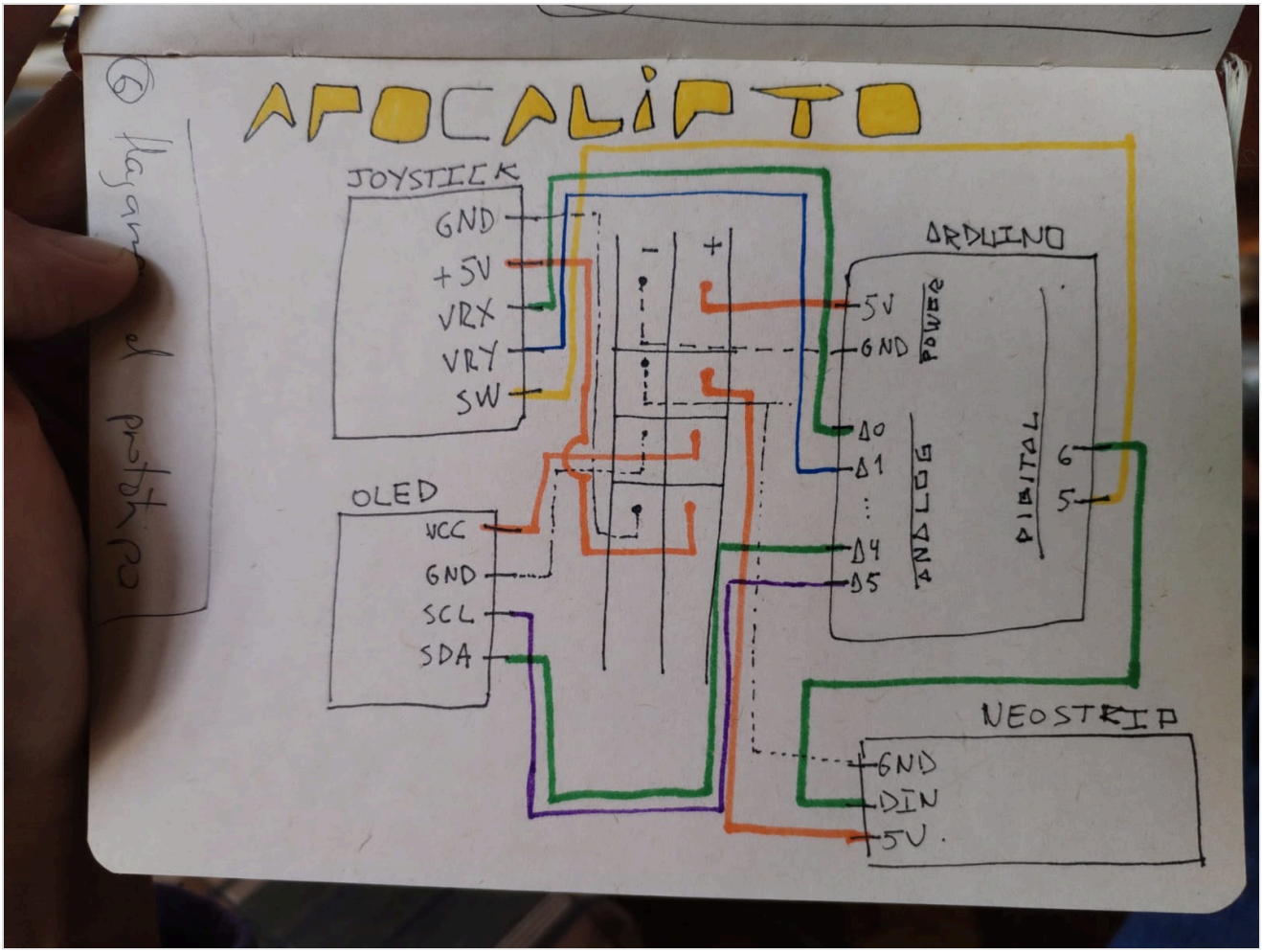
Part 4: NeoPixel Strip Connection

1. Identify the NeoPixel pins: GND, DIN, 5V
2. Connect the NeoPixel wires:

Connect GND from the NeoPixel strip to Arduino's GND pin
Connect DIN from the NeoPixel strip to Arduino's Pin 6
Connect 5V from the NeoPixel strip to Arduino's 5V pin



Full wiring diagram — joystick, OLED, and NeoPixel strip on the Arduino Uno.



Juan's original hand-drawn schematic — sketched before wiring.

Software Upload

1. Connect the Arduino to your computer via USB
2. Open the Arduino IDE
3. Install the required libraries, from Tools > Manage Libraries:
   - Adafruit NeoPixel
   - Adafruit GFX Library
   - Adafruit SSD1306
4. Open curiquingue_umbral_6.ino from the project repository
5. Select the correct board:
   - Tools > Board > Arduino Uno
6. Select the correct port:
   - Tools > Port > (select the new COM port)
7. Click the Upload button (arrow icon)

The Code

Below is the hardware setup portion of the sketch — the pin configuration and the curiquingue's real sprite, decoded byte for byte. The full, runnable file (all six game states, the maze, and the NeoPixel behavior) lives in the project's GitHub repository.

```
/*
 * ============================================================================
 * CURIQUINGUE: UMBRAL — descent + maze (Arduino Uno version)
 * GLiTCH 2026: Apocalypse Technologies — Juan Fernando Yanqui Rivera
 * ============================================================================
 *
 * Circuit: original joystick body (X/Y/SW + 22-NeoPixel strip)
 * with Elemental's 128x64 OLED screen mounted in place of the
 * damaged 128x32 screen.
 *
 * Hardware:
 *   - Arduino Uno
 *   - Analog joystick -> X: A0 (move / maze), Y: A1 (maze), SW: D2 (jump)
 *   - NeoPixel strip (22) -> D6
 *   - 128x64 SSD1306-compatible OLED, I2C -> A4 (SDA), A5 (SCL)
 *
 * GAMEPLAY
 *   Kay Pacha (descent): the curiquingue comes down from Chimborazo toward
 *   the city — joystick X moves, the SW button jumps over hazards on the
 *   ground (toxic / gas mask / radioactive). The background mountains are
 *   calculated live, nothing is saved, and pollution increases with
 *   distance traveled, not with time. 3 hits = collapse.
 *
 *   Maze (aerial): once the run's distance is reached, the view switches
 *   to a top-down plan — the curiquingue appears as a small triangle,
 *   seen from the sky. The maze is hand-designed and stored in flash
 *   (not algorithmically generated, to avoid risking the Uno's stack
 *   memory). Finding the inti glyph (the clean water source) triggers the
 *   same revelation as collapsing — two paths, one threshold.
 *
 *   Uku Pacha / Hanan Pacha / Hold / Restart: unchanged from the
 *   previous version — the strobing whiteout, the ascent, and the
 *   Andean restart rain all work exactly the same.
 *
 * HOOK FOR THE REAL TDS SENSOR (Elemental)
 *   getSpawnInterval() now receives the progress fraction (0-1) instead of
 *   elapsed time. To connect the real TDS sensor, just feed that fraction
 *   with a normalized sensor reading.
 *
 * NOTE ON SCALE
 *   The descent and the maze are deliberately simplified compared to the
 *   browser version (p5.js): no saved random levels, no real-time maze
 *   generation. The Uno has only 2KB of total RAM, and the screen alone
 *   already uses 1KB of that. Once we move to an ESP32 (520KB of RAM),
 *   we get the full procedural generation back with none of these limits.
 * ============================================================================
 */

#include <Adafruit_NeoPixel.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Fonts/Org_01.h>   // small, irregular-stroke font — used for the pacha labels

// ----------------------------------------------------------------------------
// HARDWARE CONFIG
// ----------------------------------------------------------------------------
#define LED_PIN 6
#define LED_COUNT 22

#define JOY_X A0
#define JOY_Y A1
#define JOY_SW 2

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
```

```
#define SCREEN_ADDRESS 0x3C

Adafruit_NeoPixel strip(LED_COUNT, LED_PIN, NEO_GRB + NEO_KHZ800);
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// ---------------------------------------------------------------------------
// CURIQUINGUE SPRITE — ALERT state, copied as-is from Elemental.
// 2 frames: the head turns right <-> left (32x32, 128 bytes each).
// ---------------------------------------------------------------------------
const unsigned char PROGMEM curi_alert_a[] = {
  0b00000000, 0b00000000, 0b00000000, 0b00000000,
  0b00000000, 0b00000001, 0b10000000, 0b00000000,
  0b00000000, 0b00000011, 0b11000000, 0b00000000,
  0b00000000, 0b00000001, 0b10000000, 0b00000000,
  0b00000000, 0b00000001, 0b10000000, 0b00000000,
  0b00000000, 0b00000011, 0b11000000, 0b00000000,
  0b00000000, 0b00000111, 0b11100000, 0b00000000,
  0b00000000, 0b00001111, 0b11110000, 0b00000000,
  0b00000000, 0b00011111, 0b11111000, 0b00000000,
  0b00000000, 0b00111111, 0b00111100, 0b00000000,
  0b00000000, 0b01111111, 0b11111111, 0b00000000,
  0b00000000, 0b01111111, 0b11111111, 0b11000000,
  0b00000000, 0b00111111, 0b11111110, 0b00000000,
  0b00000000, 0b00011111, 0b11111000, 0b00000000,
  0b00000000, 0b00001111, 0b11110000, 0b00000000,
  0b00000000, 0b11111111, 0b11111111, 0b00000000,
  0b00000001, 0b11111000, 0b00001111, 0b10000000,
  0b00000011, 0b11100000, 0b00000111, 0b11000000,
  0b00000111, 0b11000000, 0b00000011, 0b11100000,
  0b00000111, 0b01000000, 0b00000010, 0b11100000,
  0b00000111, 0b01000000, 0b00000010, 0b11100000,
  0b00000111, 0b01100000, 0b00000110, 0b11100000,
  0b00000111, 0b01110000, 0b00001110, 0b11100000,
  0b00000111, 0b01111000, 0b00011110, 0b11100000,
  0b00000011, 0b00111111, 0b11111100, 0b11000000,
  0b00000001, 0b00011111, 0b11111000, 0b10000000,
  0b00000000, 0b00001111, 0b11110000, 0b00000000,
  0b00000000, 0b00000111, 0b11100000, 0b00000000,
  0b00000000, 0b00000011, 0b00000000, 0b00000000,
  0b00000000, 0b00000011, 0b01100000, 0b00000000,
  0b00000000, 0b00000011, 0b01100000, 0b00000000,
  0b00000000, 0b00000111, 0b11110000, 0b00000000,
};

const unsigned char PROGMEM curi_alert_b[] = {
  0b00000000, 0b00000000, 0b00000000, 0b00000000,
  0b00000000, 0b00000001, 0b10000000, 0b00000000,
  0b00000000, 0b00000011, 0b11000000, 0b00000000,
  0b00000000, 0b00000001, 0b10000000, 0b00000000,
  0b00000000, 0b00000001, 0b10000000, 0b00000000,
  0b00000000, 0b00000011, 0b11000000, 0b00000000,
  0b00000000, 0b00000111, 0b11100000, 0b00000000,
  0b00000000, 0b00001111, 0b11110000, 0b00000000,
  0b00000000, 0b00011111, 0b11111000, 0b00000000,
  0b00000000, 0b00111100, 0b11111100, 0b00000000,
  0b00000000, 0b11111111, 0b11111110, 0b00000000,
  0b00000011, 0b11111111, 0b11111110, 0b00000000,
  0b00000000, 0b01111111, 0b11111100, 0b00000000,
  0b00000000, 0b00011111, 0b11111000, 0b00000000,
  0b00000000, 0b00001111, 0b11110000, 0b00000000,
  0b00000000, 0b11111111, 0b11111111, 0b00000000,
  0b00000001, 0b11111000, 0b00001111, 0b10000000,
  0b00000011, 0b11100000, 0b00000111, 0b11000000,
  0b00000111, 0b11000000, 0b00000011, 0b11100000,
  0b00000111, 0b01000000, 0b00000010, 0b11100000,
  0b00000111, 0b01000000, 0b00000010, 0b11100000,
  0b00000111, 0b01100000, 0b00000110, 0b11100000,
  0b00000111, 0b01110000, 0b00001110, 0b11100000,
```

```
  0b00000111, 0b01111000, 0b00011110, 0b11100000,
  0b00000011, 0b00111111, 0b11111100, 0b11000000,
  0b00000001, 0b00011111, 0b11111000, 0b10000000,
  0b00000000, 0b00001111, 0b11110000, 0b00000000,
  0b00000000, 0b00000111, 0b11100000, 0b00000000,
  0b00000000, 0b00000011, 0b00000000, 0b00000000,
  0b00000000, 0b00000011, 0b01100000, 0b00000000,
  0b00000000, 0b00000011, 0b01100000, 0b00000000,
  0b00000000, 0b00000111, 0b11110000, 0b00000000,
};

// (Full game logic — states, drawing, NeoPixel behavior, and the maze —
//  continues below. See the companion GitHub repository for the complete,
//  runnable file: curiquingue_umbral_6.ino)
```

How This Program Works

This sketch runs the whole game as a state machine: at any moment, the Arduino is in exactly one of six states, and each loop() call only draws and updates whichever state is currently active.

1. Kay Pacha — the descent. The curiquingue walks toward the city, jumping over hazards.
2. Maze Pacha — the aerial maze, seen from above, searching for clean water.
3. Uku Pacha — the threshold. A strobing whiteout, triggered by either collapse or finding the water.
4. Hanan Pacha — the ascent. The bird rises, whole again.
5. Hold Pacha — a held breath at the top of the ascent.
6. Restart Transition — the glyph rain, before the cycle loops back to Kay Pacha.

Key Concepts

1. State machine

The game's structure is one enum (GameState) and one switch statement in loop(). Nothing runs outside of its assigned state — this keeps the logic easy to follow even as the story gets more elaborate.

2. Analog input

The joystick reports two continuous values (0–1023), read exactly the way a potentiometer is read. Moving it up on the Y axis is what triggers a jump — a small, deliberate deviation from using the click button.

3. Memory constraints

The Arduino Uno has only 2KB of RAM, and the OLED buffer alone uses about half of that. Because of this, the maze is hand-drawn once and stored in flash memory (PROGMEM) instead of being generated by an algorithm — a real recursive maze generator would risk the little stack memory the board has left.

4. Timing without delay()

Instead of pausing the whole program between phases, the code checks millis() to know how much time has passed. This keeps the joystick responsive at every moment, even during timed sequences like the strobe or the glyph rain.

5. PROGMEM sprites

The curiquingue's bitmap is stored in flash memory rather than RAM, and read back with pgm_read_byte() whenever it needs to be drawn. This is the same technique used for the maze layout, and it is what makes the

sketch fit on the Uno at all.

Full source code, wiring diagrams, and updates: see the project's GitHub repository.